

code# Notas de clases de PROGRA II

Aether / MeganHolic (Nombre de mi repositorio)

-apuntes de las clases paso a paso

Class I : Git

Primer programa en Java

- 461 cd ..
- 462 ls -la
- 463 cd prjGit
- 464 ls -la
- 465 code . : para abrir mi repositorio
- 466 touch Hi.java Sumar.java : para crear documentos con touch
- 467 ls
- 468 rm Hi.java Sumar.java : elimine aca los dos documentos porque tenia repetidos
- 469 cd ..
- 470 cd prjGithub : fui a mi repositorio a crear las nuevas clases
- 471 touch Hi.java Suma.java : cree los nuevos documentos
- 472 cd Hi.java
- 473 code Hi.java : para ingresar a mi archivo para editar
- 474 history : para ver mis lineas de codigo

Mi programa de Hello World

```
public class Hi{  
  
    public static void main(String[] args) {  
        System.out.println("Hi MeganHolic, welcome to Java programming!");  
        System.out.println("Hi MeganHolic, welcome to Java programming!");  
        System.out.println("Im thunder ");  
        System.out.println("Im thunder ");  
    }  
}
```

Mi programa de Suma en Java

- 480 clear
- 481 code Suma.java
- 482 javac Suma.java
- 483 java Suma
- 484 java Suma
- 485 java Suma

- 486 history+
- 487 history
- \$ pwd
- /c/MeganHolic/progra_two/prjGithub

```
public class Suma{  
  
    public static void main(String[] args) {  
        int num1 = 7;  
        int num2 = 770;  
        int sum = num1 + num2;  
        System.out.println("la suma de dos numeros " + num1 + " y " + num2 + " es:  
" + sum);  
    }  
}
```

Class II : github

Comandos Linux

- 441 ls -ña
- 442 ls -la
- 443 pwd : ubicacion de mi directorio o repositorio
- 444 pwd
- 445 cd c:/ ir a una ruta con cd
- 446 ls -la para enlistar con archivos protegidos o ocultas
- 447 cd MeganHolic/
- 448 ls -la
- 449 wsl llamar a la maquina virtual
- 450 exit
- 451 java Mate. con tab puedo ver los documentos con las características
- 452 java Mate.java
- 453 java Mate.java
- 454 java Mate.java
- 455 git status
- 456 exit
- 457 touch readme.md
- 458 echo "#Class II : github" >> readme.md
- 459 code readme.md
- 460 history >> readme.md

Como conectar el repositorio con git

480 git remote -v

- 481 git remote set-url origin https://github.com/aerher/Prueba-Repositorio.git : conectamos el repositorio al destinatario para guardar los siguientes git pull
- 482 git push -u origin main
- 483 git status
- 484 .gitignore
- 485 code .gitignore
- 486 git rm --cached *.class : eliminamos los documentos que esten de mas en nuestro proyecto y validando el gitingnore
- 487 git rm --cached *.md
- 488 git status
- 489 git add. : agrego los nuevos cambios
- 490 git add .
- 491 git commit -m "Eliminando los archivos del gitignore" : comento lo que realice
- 492 git push : subo a mi nube de Github con los cambios realizados
- 493 history

Git comand

comandos base

- git clone : voy a traer un repositorio de la nube a mi maquina con su URL

1 vez

- git init : para primera vez para darme noticias sobre mi repositorio o iniciar mi proyecto
- gitignore : para no registrar documentos privados Repetitivamente / cada dia
- git status : el estado de mi repositorio
- git add . : para agregar todo
- git add xxx/ . : es para agregar o quitar cambios
- git comit -m "(podemos poner mensaje aqui para su cambio guardado)" : nombre que voy a guardar mi archivo

para subir los cambios de la organizacion

- git push : enviar todo a mi nube de GitHub
- ls -a para llamar a todos los documentos sin restrinciones
- cat : es para revisar documentos que esten guardados o destinados a llevar autoguardado.
- .md : para crear documentos de texto

Conectar repositorios local a remoto en GITHUB

- git init
- git status

- git add .
- git commit -m ""
- git push

Pero si git push no conectado a un url

- git remote add origin "URL html " : nos sirve para conectar el repositorio virtual con nuestra maquina local y se puedan hacer los push de nuestros archivos
- git push -u origin main : nos sirve para declarar por primera vez el update de nuestro repo a GITHU , demas con " - u " nos sirve para simplificar el hecho de repetir el " origin main " sino que solo con git push, ya nos dejaria subir nuestro repo local.

• Comandos para Markdown

- **![titulo de la imagen a presentar](colocar ruta especifica de la imagen):** es para presentacion de imagenes en un markdown y que se exporte a un pdf para presentacion formales

Actividad 13/04

- git log -d--decorate --oneline : es para saber cuando mcomit he hecho sobre mi proyecto y ver cambios y modificaciones
- git tag <"name "> : es para darte una etiqueta a un commit que sea importante de todos los commit que he realizado
- git branch : para ver las ramas que tengo de mi git o repositorio que he ido haciendo commit

Introducion en Java

******es una de los IDE mas famosos que existe actualmente, que puede requerir para muchos problemas o resolucion de programas como paginas web , aplicaciones, etc.

una aplicacion que tiene soporte en todo el mundo, principalmente es un IDE que se orienta en su programacion en objetos.

es una buena alternativa para otros IDEs que son conocidos como C ++ , etc. ******

Esqueleto De Un Programa En Java

las primeras lineas de codigo como ejm 1 y 2, es para decalaracion de un package o una libreria a llamar, como un import.

- package : en que parte o en donde se va a agrupar mi codigo en mi repositorio. donde podemos agregar varios codigos en un mismo package y que no sea tan dificil ubicarlas entre si.
- class : es para nombre a nuestro documento y donde se ejecuta el codigo si o si .
- variables : caracteres que nosotros definimos en nuestro codigo como (int, float, double, Strin)
- public, private, protected : Son Moderadores de acceso a nuestro codigo o en este claso nuestra clase a construir.
- main : es el metodo principal en java, como decir el codigo a ejecutar.
- new : es para crear en nuestro (Main) podemos crear un objeto de tipo de dato especifico
- ; : siempre se utiliza para cerrar una linea de codigo, ya que sino da fallos en su complilacion
- {} : para abrir bloques de codigo en nuestro codigo e identificarlo.



POO



ICOM